

# Publishing spec — the human approval gate (Lab 10)

Multi AI Agents Workflow for Content Creation (ChatGPT Edition) · Day 2 · Use Codex

Nothing reaches LinkedIn, Facebook or YouTube unless a **person** approved

that exact version.

## Content files

One file per item: content/<channel>/<week-or-issue>/<id>.md

```
---
id: li-w01-01
channel: linkedin          # linkedin | facebook | youtube | newsletter
title: Know your number in 30 minutes
link: https://tertiarycourses.github.io/TGS-2023036153-Multi-AI-Agents-Workflow-for-Content-
Creation/?utm_source=linkedin&utm_medium=social&utm_campaign=know-your-number
image: content/social/week-01/li-w01-01.png  # linkedin, facebook: PNG, JPG or GIF
alt: Sunny the sun beside the words Know your number  # the image's alt text
video: content/video/ep01/ep01.mp4         # youtube only
tags: retirement, cpf, planning             # youtube only
---
The post text (for YouTube: the video description).
```

## review/approvals.csv

id,channel,file,sha256,status,submitted\_by,approver,approved\_at,notes

- status: pending | approved | changes | rejected
- sha256: hash of the content file, plus its image (LinkedIn, Facebook) or

video (YouTube) — a new picture after approval needs a new approval.

## Scripts (Node 22, no packages)

| Script                                                 | Who runs it                             | What it does                                                                                                                                                                                                                  |
|--------------------------------------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| scripts/submit.mjs <file>                              | agents                                  | Adds or refreshes a <b>pending</b> row with the file's hash                                                                                                                                                                   |
| scripts/approve.mjs <id> --by "Name" [--changes "why"] | a <b>person</b> , in their own terminal | Approves (or sends back) the CURRENT version: records approver, time and hash. Refuses to run without an interactive terminal.                                                                                                |
| scripts/publish.mjs <id> [--live]                      | agents or people                        | Dry run by default: prints the exact request, sends nothing. With --live: refuses unless the row is approved AND the hash still matches; refuses an id already in review/publish-log.csv (no double posts); logs the post id. |

## The hook (Codex, .codex/hooks.json — trust it first)

A PreToolUse hook on shell commands runs scripts/gate-hook.mjs:

- blocks any agent command that runs approve.mjs (only people approve);
- blocks agent edits to review/approvals.csv;
- blocks publish.mjs ... --live unless the item is approved and unchanged.

It blocks with exit code 2 and a reason the agent can read.

## The APIs

**LinkedIn** — POST <https://api.linkedin.com/rest/posts>

Headers: Authorization: Bearer <token>, LinkedIn-Version: <YYYYMM>,

X-Restli-Protocol-Version: 2.0.0, Content-Type: application/json.

Body: author (urn:li:person:...), commentary, visibility PUBLIC,

distribution {feedDistribution MAIN\_FEED}, lifecycleState PUBLISHED.

Needs the w\_member\_social permission. The post id comes back in the x-restli-id header.

With an image, first POST <https://api.linkedin.com/rest/images?action=initializeUpload>

with {"initializeUploadRequest": {"owner": "<author URN>"} }; PUT the image

bytes to the returned uploadUrl (with the Authorization header); then add

content {media {id: <the returned image URN>, altText}} to the post.

**Facebook Page** — POST <https://graph.facebook.com/<version>/<page-id>/feed>

with message, link and a Page access token (pages\_manage\_posts,

pages\_read\_engagement). With an image, POST <page-id>/photos instead as

multipart/form-data: source (the image file), caption (the text and link),

alt\_text\_custom; the feed post id comes back as post\_id.

**YouTube** — resumable upload:

POST [https://www.googleapis.com/upload/youtube/v3/videos?](https://www.googleapis.com/upload/youtube/v3/videos?uploadType=resumable&part=snippet,status)  
[uploadType=resumable&part=snippet,status](https://www.googleapis.com/upload/youtube/v3/videos?uploadType=resumable&part=snippet,status)

with snippet (title, description, tags, categoryId 27) and status

(privacyStatus **private**, selfDeclaredMadeForKids false), then PUT the

video bytes to the returned Location. Get an access token from the refresh

token at <https://oauth2.googleapis.com/token>. Uploads from an unaudited API project stay private — a person makes the video public in YouTube Studio.

**Newsletter** — no API. Created as files and Gmail drafts (Lab 11).